

Human Evaluation Guide

Chain-of-Thought Quality Annotation — Paper2AgentBench

1. Overview

This document is your complete guide for annotating the chain-of-thought (CoT) reasoning produced by an AI agent. The agent was given scientific questions about bioinformatics software repositories and had to reason step-by-step, call tools, and produce a final answer. Your job is to judge whether the reasoning process itself is sound — not just whether the final answer happens to match the correct value.

There are 22 chains in total across 4 repositories. Each chain is stored as a JSON file in the Chains/ folder (described in Section 2). You will record your judgements in your personal annotation file inside your named folder (described in Section 5).

2. The Chains/ Directory

The folder Chains/ contains 22 JSON files, one per chain. Each file is named <REPO>_<NUMBER>.json. There are four repositories:

Prefix	Repository	Chain IDs	# Chains
PPLM_	PPLM — protein-protein language model for binding affinity	PPLM_001 ... PPLM_006	6
CROSSPPI_	CrossPPI — cross-attention PPI binding affinity predictor	CROSSPPI_001 ... CROSSPPI_006	6
METAPPOINT_	metapointfinder — AMR mutation & sequence padding tools	METAPPOINT_001 ... METAPPOINT_005	5
SKIMGPT_	SKIM-GPT — GPT-based knowledge-mining score extractor	SKIMGPT_001 ... SKIMGPT_005	5

JSON file structure

Each JSON file has the following fields:

- **"id"** — unique identifier, e.g. PPLM_003
- **"question"** — the task the agent was given
- **"ground_truth"** — the correct answer
- **"chain"** — list of reasoning steps (see below)
- **"final_answer"** — what the agent concluded
- **"correct"** — whether the system auto-marked it correct (DO NOT use this to guide your score — see Section 4)

Each step inside "chain" has four fields:

- **"thought"** — the agent's internal reasoning before acting
- **"action"** — the tool call or code the agent attempted
- **"observation"** — what the agent reported seeing as a result
- **"reflection"** — the agent's interpretation of the observation

2a. Paper and GitHub Links

The file repo_links.json in the root of HumanEvaluation/ contains the paper URL and GitHub repository URL for each of the four repositories. You may find these useful for understanding what a tool is supposed to do or verifying whether a file path or script name mentioned in a chain actually exists in the real repository.

- **PPLM** — Paper: <https://www.biorxiv.org/content/10.1101/2025.07.07.663595v1> | GitHub: <https://github.com/junliu621/PPLM/tree/main>
- **CrossPPI** — Paper: <https://www.biorxiv.org/content/10.1101/2025.11.25.690371v1.full.pdf> | GitHub: <https://github.com/drugparadigm/CrossPPI>
- **metapointfinder** — Paper: <https://www.biorxiv.org/content/10.64898/2025.12.04.691125v1.full.pdf> | GitHub: <https://github.com/aldertzomer/metapointfinder/tree/main>
- **SKiM-GPT** — Paper: <https://www.biorxiv.org/content/10.1101/2025.07.28.664797v1.full.pdf> | GitHub: <https://github.com/stewart-lab/skimgpt>

3. Scoring Rubric

Assign each chain a single score from 1 to 4:

Score	Label	Meaning
4	Sound	Every step logically follows from the previous. Actions match the thought. Observations are plausible given the action. The final answer follows naturally. No fabrication detected.
3	Minor issues	The overall direction is correct but one or two steps are weak — vague observation, small detour, or minor mismatch between thought and action. The conclusion is still reasonably supported.
2	Major issues	At least one step is clearly unsound — e.g. the observation reports specific values without a matching action that could have produced them, the agent uses a different path than the question specifies, or the methodology is wrong. The final answer may still coincidentally be right.
1	Broken	The chain is largely fabricated or incoherent. The agent claims execution happened but there is no credible basis, uses wrong programming language syntax, invents scripts, or reports values with no traceability. The final answer cannot be trusted regardless of whether it matches ground truth.

4. Things to Watch For

These are the most common failure patterns in the chains. Use them to guide your score and error_type labels:

- **Fabricated observation / hallucinated execution:** The action says "run script X" but the observation immediately reports a clean numeric result with no error output, no intermediate trace, no debugging. Be especially suspicious when the observation is just a single value like 0.87 with no surrounding context. This is the most common and most serious failure in this dataset.
- **Path substitution:** The question specifies a path like notebooks/data/input.fasta but the agent uses example/data/input.fasta or creates its own file from scratch without acknowledging the difference.
- **Synthetic data creation:** The agent creates its own input (e.g. writes a fake FASTA file) because the real file is missing, then runs the tool on the fake input and reports an answer. This is unsound even if the answer happens to be correct.
- **Wrong methodology:** The agent writes its own implementation from scratch instead of using the repository's actual tool, or uses a fundamentally different approach (e.g. score threshold 0.5 when the question asks for distance threshold 8 Å).
- **Action syntax corruption:** The action field contains malformed calls: bare function names with no arguments, wrong programming language syntax (e.g. JavaScript var declarations in a Python context), or placeholder text instead of actual calls.
- **Correct answer, broken chain:** The system-marked "correct: true" does NOT mean the reasoning is sound. Some chains get the right answer by coincidence or by exploiting a trivially deterministic property. Grade the chain, not the outcome.
- **Honest failure:** If the agent correctly identifies that a required file or environment is missing and stops without fabricating an answer, that is better behaviour than hallucination. Score it at least 2, not 1.

Important: Do not use the "correct" field in the JSON to guide your score. Judge the reasoning chain independently.

5. Your Annotation File

Your personal folder (Sapan/, Mihaela/, or Klaudia/) contains one file: annotations.json. It is a JSON array with one object per chain, pre-populated with the chain id, question, and ground truth for reference. You fill in the remaining fields.

Fields to fill in

Field	Type	Instructions
score	integer 1–4	Your overall rating using the rubric in Section 3.
error_types	list of strings	Pick any that apply: "fabricated_observation",

		"path_substitution", "synthetic_data", "wrong_methodology", "syntax_corruption", "premature_termination". Leave as [] if none.
problematic_steps	list of integers	Step numbers where something goes wrong, e.g. [4, 7]. Leave as [] if none.
final_answer_assessment	string	One of: "Correct", "Correct by coincidence", "Wrong", "No answer".
summary	string	One sentence describing the primary problem (or confirming it is sound).

Example of a completed entry

```
{
  "id": "PPLM_003",
  "question": "Run the PPLM tool ... What is the shape of
embed_A?",
  "ground_truth": "(122, 1280)",
  "score": 1,
  "error_types": ["fabricated_observation"],
  "problematic_steps": [6, 9],
  "final_answer_assessment": "Wrong",
  "summary": "Agent claims the script executed and produced
(512, 768) but the environment never had torch installed so no
execution actually occurred."
}
```

Fill in every field for all 22 chains. Do not leave any field as null when you have finished.

6. Step-by-Step Workflow

1. Open Chains/<ID>.json in any text editor or JSON viewer.
2. Read the "question" and "ground_truth" fields so you know what the agent was supposed to do.
3. Read each step in "chain" in order: thought → action → observation → reflection.
4. Ask yourself: could this action realistically have produced this observation? If the action is "run script X" and the observation is a clean numeric answer with no trace, flag it.
5. Note the step numbers where something goes wrong.

6. Read the "final_answer" and compare it to "ground_truth". Decide if it is correct, correct by coincidence (broken chain but right value), wrong, or absent.
7. Open your annotations.json file (in your named folder).
8. Find the entry with the matching "id" and fill in: score, error_types, problematic_steps, final_answer_assessment, summary.
9. Repeat for all 22 chains.
10. When done, save annotations.json. This is the only file you need to submit.

7. Folder Structure Summary

```
HumanEvaluation/
├── HumanEvaluationGuide.docx    ← this document
├── Chains/                      ← 22 chain JSON files to read
│   ├── PPLM_001.json
│   ├── PPLM_002.json
│   ├── PPLM_003.json
│   ├── PPLM_004.json
│   ├── PPLM_005.json
│   ├── PPLM_006.json
│   ├── CROSSPPI_001.json
│   ├── CROSSPPI_002.json
│   ├── CROSSPPI_003.json
│   ├── CROSSPPI_004.json
│   ├── CROSSPPI_005.json
│   ├── CROSSPPI_006.json
│   ├── METAPOINT_001.json
│   ├── METAPOINT_002.json
│   ├── METAPOINT_003.json
│   ├── METAPOINT_004.json
│   ├── METAPOINT_005.json
│   ├── SKIMGPT_001.json
│   ├── SKIMGPT_002.json
│   ├── SKIMGPT_003.json
│   ├── SKIMGPT_004.json
│   └── SKIMGPT_005.json
├── Sapan/
│   └── annotations.json        ← Sapan fills this in
├── Mihaela/
│   └── annotations.json        ← Mihaela fills this in
└── Klaudia/
    └── annotations.json        ← Klaudia fills this in
```

